



ChildTaskTracker

Turning household chores into a habit kids actually want to keep — and parents can trust.

MVP DEMO 2026-09-16 childtasktracker.github.io

Sticker charts don't scale to two parents and a wide age range.

Most families track chores with a whiteboard or an app that stops working the moment a second parent, a teenager, or a real reward system gets involved.



No shared visibility

One parent logs it, the other is out of the loop — no easy way for both to see and approve.



One-size-fits-all

The same sticker chart doesn't motivate a 7-year-old and a 15-year-old equally.



Rewards feel arbitrary

No system connecting effort to reward — it's negotiated fresh, every single time.

02 THE SOLUTION

One loop: a kid marks it done, a parent confirms it, points do the rest.

ChildTaskTracker turns chores into a shared, validated, rewarding routine — without becoming another thing parents have to administer by hand.



Scoped deliberately: one family, real validation, AI as a helper — never the decision-maker.

Who it's for

- ✓ Co-parents sharing validation duties
- ✓ Kids ages 6–16, with age-adaptive tone
- ✓ Single-family tool first — built to generalize later

Non-negotiables

- ✓ AI recommends only from the parent's own reward list
- ✓ Every AI feature has a deterministic fallback
- ✓ 24h auto-approval, one-day streak forgiveness

Three directions, prototyped as real landing pages — then one clear pick.



A • Calm & minimal

Faint pastel washes, quiet confidence, no gamification chrome.



B • Warm & playful ✓ chosen

Duolingo-style energy — bright, bold, mascot-led. Best fit for ages 6–16.



C • Lively & animated

Garden-path metaphor with confetti and motion — maximalist, high build cost.

A fully working demo – not a mockup. Every flow below actually runs.

Child side

- ✓ Tasks tab — tap to mark done, confetti on completion
- ✓ Rewards shelf — redeem once points reach the threshold
- ✓ Progress — streak, points, AI weekly recap

Parent side

- ✓ Validate queue — approve/reject, live 24h countdown
- ✓ Manage children, tasks, rewards, and names
- ✓ Insights — AI reward suggestion + KPI summary

0

BACKEND SERVERS

4

FILES, TOTAL

100%

FUNCTIONAL OFFLINE / NO KEY

Everything lives in the browser. Only one optional call ever leaves it.

HOSTING

GitHub Pages, static files

STATE

One JS object + localStorage

UI

Vanilla JS, hash router, no
build step

OPTIONAL

Direct OpenAI call, key stays
local

Every AI feature — reward suggestions, weekly recaps, starter tasks — has a rule-based fallback. No key, no network, no problem: the app behaves identically either way.

A 6-module interactive course, built from this app's own real code.

Not slides about the code — the actual functions, the actual bugs, walked through with live quizzes and animations. No coding background required.



1 • What Happens When You Tap

One click traced end-to-end, into a group-chat animation of both parents getting notified.



2 • Meet the Cast

The four files and the one shared STATE object everything reads and writes.



3 • The Loop

Click → mutation → state → localStorage/render — the same five steps, every time, animated.



4 • Calling In Outside Help

The one call that leaves the browser, and the guardrail that keeps the AI on a leash.

08 KEY TAKEAWAYS

What building this actually taught us.

-  **One line broke every modal button**
An `event.stopPropagation()` silently killed every Save/Send button — caught only by live-clicking the deployed app, not by reading the code.
-  **CDNs hide your own fixes**
GitHub Pages' edge cache kept serving old JS after a push — versioned URLs (`app.js?v=4`) made re-verification trustworthy.
-  **Fallback-first beats AI-only**
Every AI feature ships a rule-based twin — doubles the code, but the product never breaks when a key is missing or the API is down.
-  **The real tradeoff: no backend, no sync**
Zero infrastructure was the right call for an MVP demo — but it means no real cross-device sync until a backend is added.

09 SEE IT LIVE

Everything from this deck, one tap away.



Live app

adamhelwa.github.io/ChildTaskTracker



Source + PRD + roadmap + architecture

github.com/adamhelwa/ChildTaskTracker



Interactive "How It Works" course

adamhelwa.github.io/ChildTaskTracker/course



Let's see it in action.

Questions welcome — and yes, you can tap the real buttons.